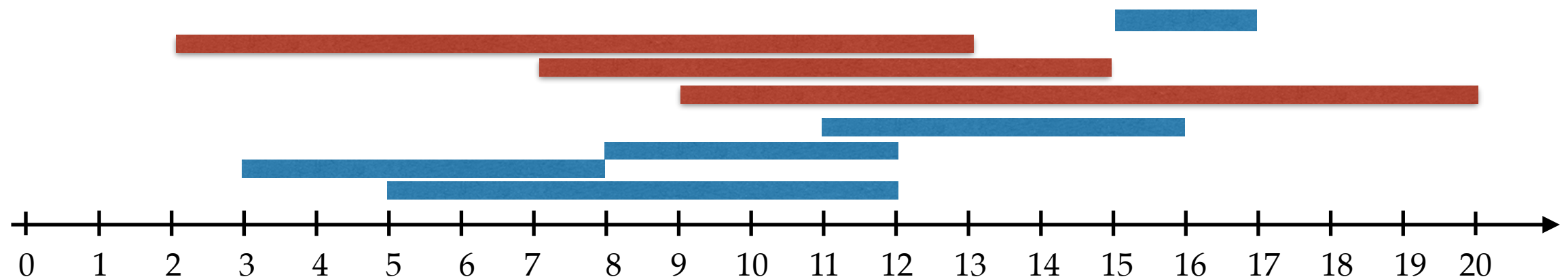


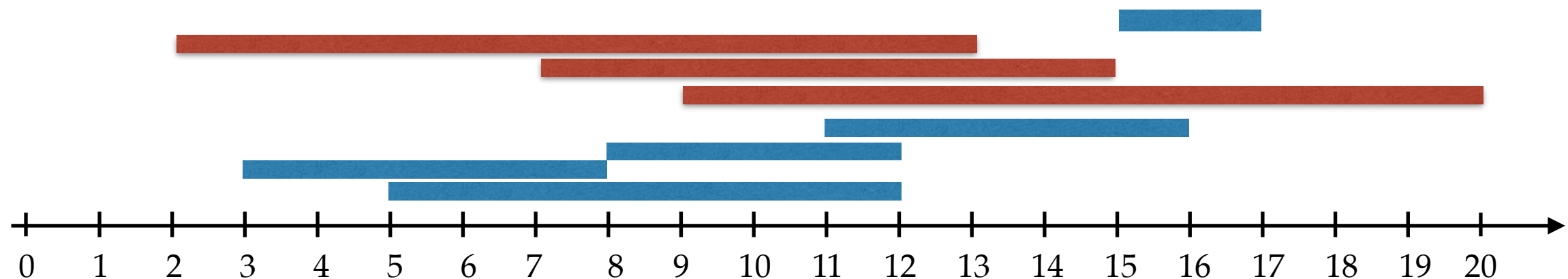
Compute ONLY
What's Necessary!?!...

Interval containment



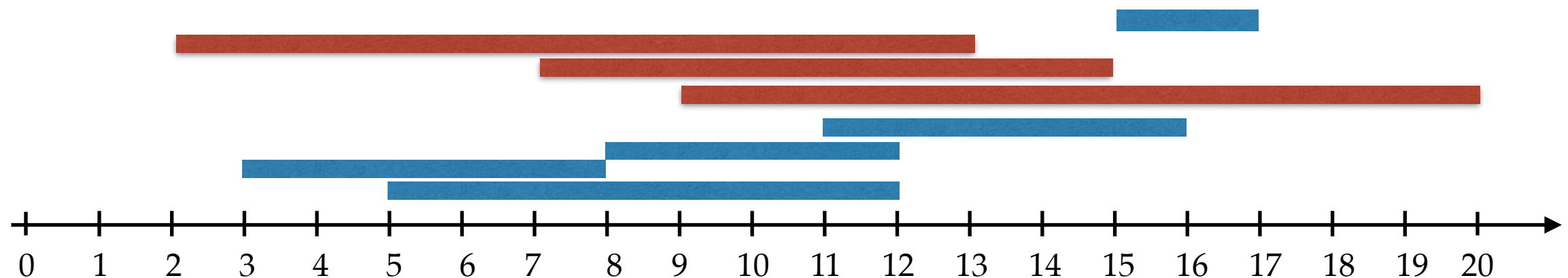
Interval Containment

- Problem: Given a set of intervals count the number of intervals which are not contained in any other interval.
- $(5, 12)$, $(3, 8)$, $(8, 12)$, $(11, 16)$, $(9, 20)$, $(15, 17)$, $(7, 15)$, $(2, 13)$
- $(9, 20)$, $(7, 15)$, $(2, 13)$



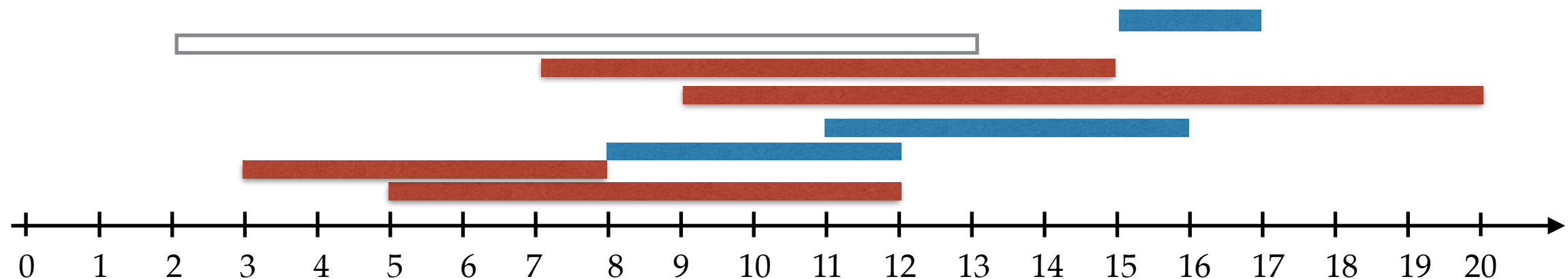
Interval containment

- Given a set of intervals
count the number of intervals which are not contained in any other interval.
- Naive solution: for every interval, check every other interval
- $O(n^2)$



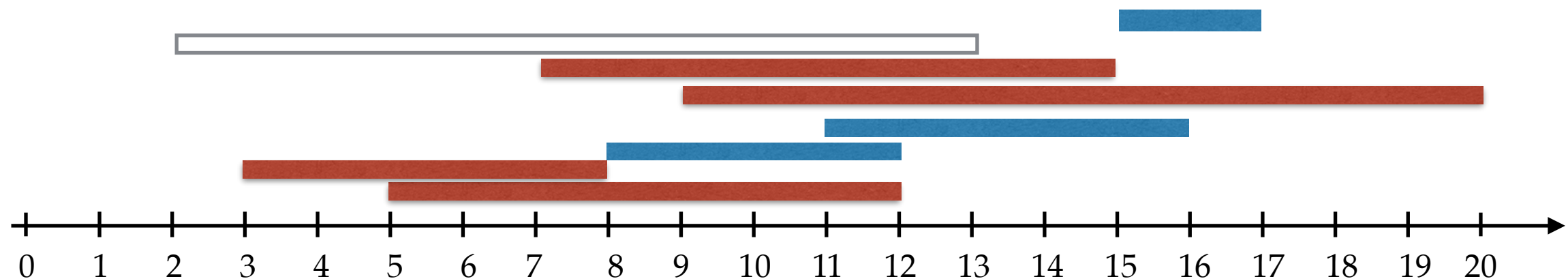
Subproblem

- Suppose we have a solution for first $n-1$ intervals.
- $(5, 12), (3, 8), (8, 12), (11, 16), (9, 20), (15, 17), (7, 15), \dots$
- $(3,8), (5, 12), (9, 20), (7, 15)$



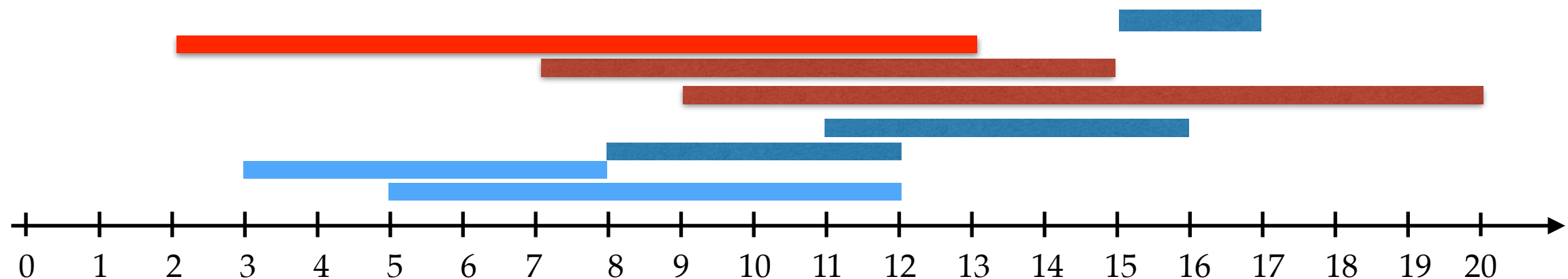
Subproblem

- Suppose we have a solution for first $n-1$ intervals.
- $(5, 12), (3, 8), (8, 12), (11, 16), (9, 20), (15, 17), (7, 15), (2, 13), \dots$
- $(3,8), (5, 12), (9, 20), (7, 15)$



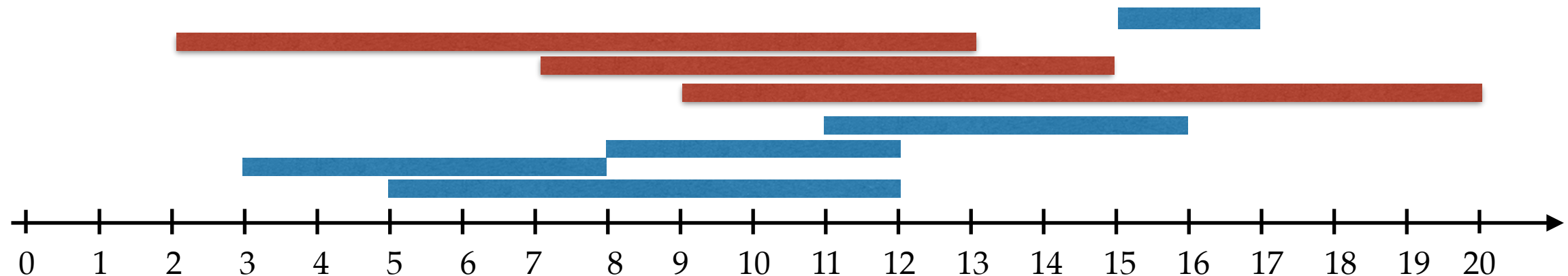
Interval containment

- When we introduce the n th interval need to check if it is contained in any other interval or if it contains other intervals. Seems to take $O(n)$ time.
- (5, 12), (3, 8), (8, 12), (11, 16), (9, 20), (15, 17), (7, 15), (2, 13)
- ~~(3, 8)~~, ~~(5, 12)~~, (9, 20), (7, 15), (2, 13)



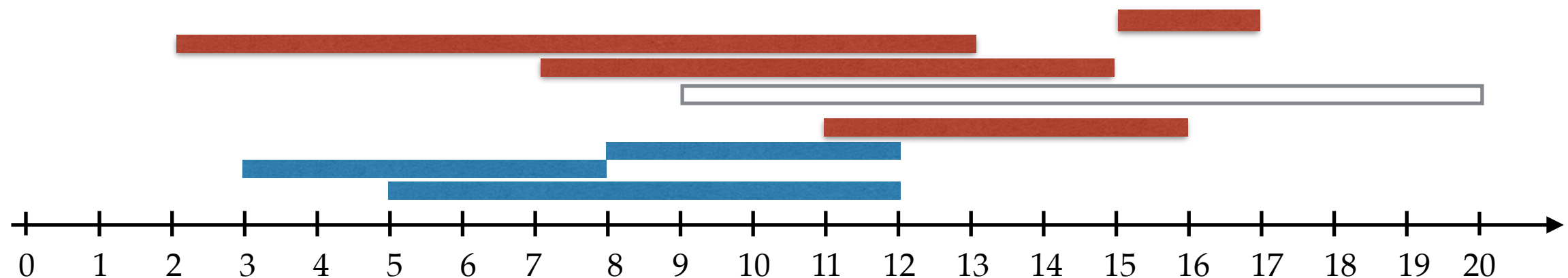
Reordering input

- Can we reorder the intervals so that the work at the last step reduces?
- Two possible orders: increasing start time, increasing finish time
 - (2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),
 - (3, 8), (5, 12), (8, 12), (2, 13), (7, 15), (11, 16), (15, 17), (9, 20),



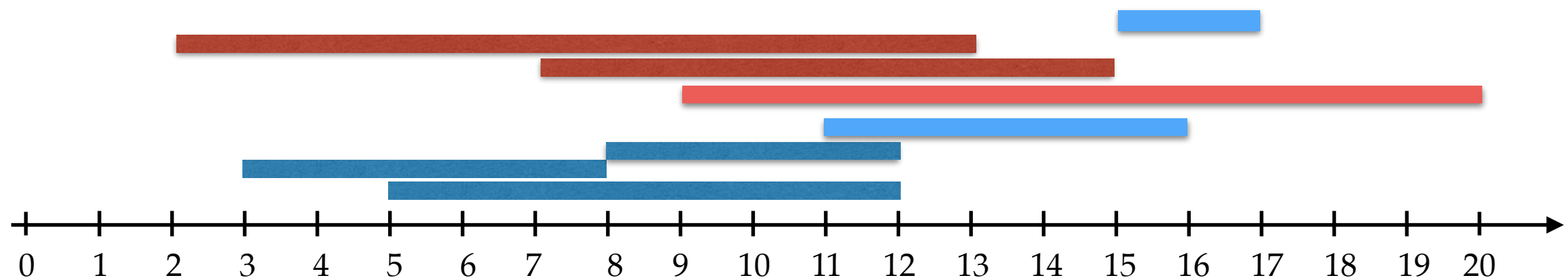
Increasing finish time

- Can we reorder the intervals so that the work at the last step reduces?
- Consider increasing finish time
 - $(3, 8)$, $(5, 12)$, $(8, 12)$, $(2, 13)$, $(7, 15)$, $(11, 16)$, $(15, 17)$, $(9, 20)$,
 - $(2, 13)$, $(7, 15)$, $(11, 16)$, $(15, 17)$,



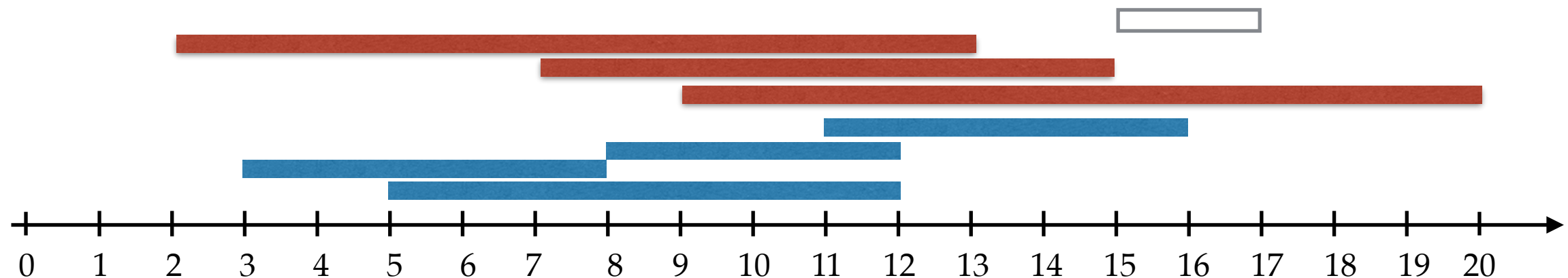
Increasing finish time

- Can we reorder the intervals so that the work at the last step reduces?
- Consider increasing finish time. Same $O(n)$.
 - (3, 8), (5, 12), (8, 12), (2, 13), (7, 15), (11, 16), (15, 17), (9, 20),
 - (2, 13), (7, 15), ~~(11, 16)~~, ~~(15, 17)~~, (9, 20),



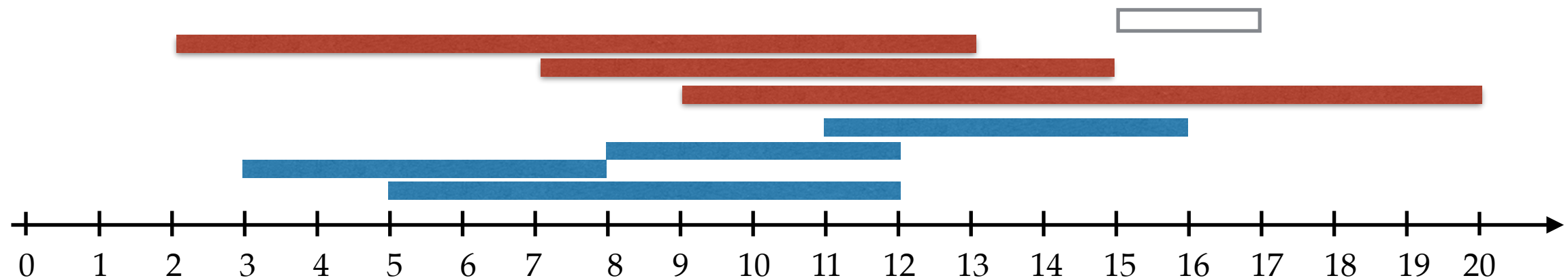
Increasing start time

- Consider increasing start time
 - $(2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),$



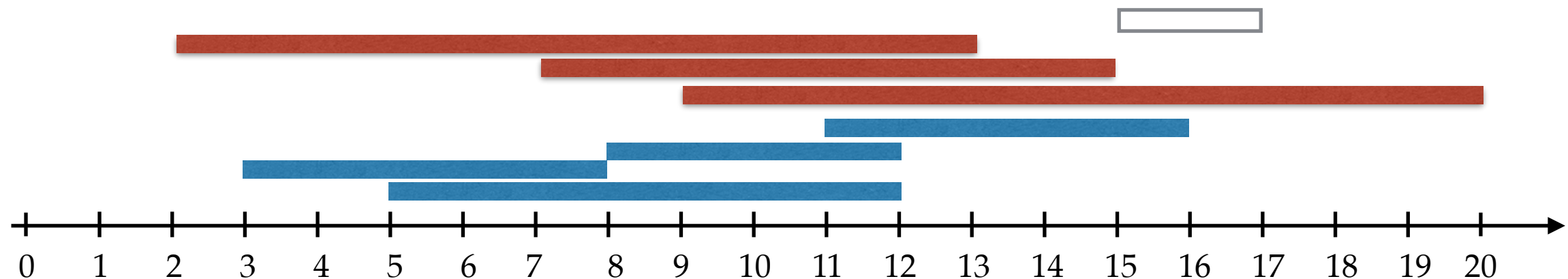
Increasing start time

- Consider increasing start time
 - $(2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),$
 - Observation: The last interval cannot contain any other



Increasing start time

- Consider increasing start time
 - $(2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),$
 - **Observation:** The last interval cannot contain any other
 - Need to check whether the last interval is contained in any other
 - (Same as whether any previous interval finishes after the last one)...



Algorithm

- Maintain the highest finish time among interval seen so far
- Go over all intervals in increasing order of start times.

Algorithm

- Maintain the highest finish time among interval seen so far
- Go over all intervals in increasing order of start times.
- For an interval (s_i, f_i) :
 - if $f_i > largest_finish_time$ then
 - $number_of_maximal_intervals ++$;
 - $largest_finish_time \leftarrow f_i$

Algorithm

- Maintain the highest finish time among interval seen so far
- Go over all intervals in increasing order of start times.
- For an interval (s_i, f_i) :
 - if $f_i > largest_finish_time$ then
 - `number_of_maximal_intervals ++ ;`
 - $largest_finish_time \leftarrow f_i$
- *$O(n \log n)$ time...*

Ideas so far

- Reducing to a subproblem
- **Stronger induction hypothesis:** Sometimes may need to solve more than what is asked for
- Reordering the input can be helpful.

Other Problems on Intervals

Problem 1: Given n intervals, find the total length of their union.

Problem 2: Given n intervals, find the number of intervals which do not overlap with any other intervals.

Problem 3: Given n intervals, find the maximum number of mutually intersecting intervals.

Divide and Conquer

Divide & Conquer

- 1. **Divide** the problem into smaller subproblems
- 2. **Conquer** the subproblems via recursive calls
- 3. **Combine** the subproblem answers...

An Intuitive Viewpoint

- Break the problem into smaller subproblems...
- Assume that some 'oracle' solves the smaller subproblems and gives you the solution
- Try to solve the original problem using these solutions to subproblem...

An Intuitive Viewpoint

- Break the problem into smaller subproblems...
- Assume that some 'oracle' solves the smaller subproblems and gives you the solution
- Try to solve the original problem using these solutions to subproblem...
- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

An Intuitive Viewpoint

- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !
- But where is the oracle ? There is none !
- Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)...

An Intuitive Viewpoint

- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !
- But where is the oracle ? There is none !
 - Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)...
 - The recursion paradigm keeps combining solutions to smaller subproblems starting with the base cases...

Divide and Conquer

- Classic example: merge sort
 - Input: 1, 3, 5, 2, 4, 6; Output: 1, 2, 3, 4, 5, 6
 - Naive approach could take $O(n^2)$ time.
 - Divide the problem into two (or more) subproblems of size $n/2...$

Divide and Conquer

- Classic example: merge sort
 - Divide the problem into two (or more) subproblems of size $n/2$
- Combine the solutions of the subproblems and build a solution for the original problem
- $T(n) = a T(n/2) + f(n)$
- Divide and conquer might give a running time improvement e.g., from $O(n^2)$ to $O(n \log n)$.

Divide and Conquer

- Counting Inversions in an Array:
 - Input: An array A of (distinct) natural numbers, suppose $1, 2, 3, \dots, n$ (for this lecture) in any arbitrary order.
 - Output: The number of inversions in the array = number of pairs (i, j) such that $i < j$, but $A[i] > A[j]$.
 - Example: $1, 3, 5, 2, 4, 6, \dots$
 - # of inversions = 3
 - $(3, 2), (5, 2), (5, 4)$

Divide and Conquer

- Application: E-commerce Websites : Collaborative filtering... Inversion counting is used to determine 'similarity' between two customers...
- Naive algorithm takes $O(n^2)$ time.
- Can we design an $O(n \log n)$ time algorithm, which outputs all inversions?

Divide and Conquer

- Suppose A is divided into X and Y .
- Left inversion : Both (i,j) in X
- Right inversions : Both (i,j) in Y
- Split inversions : i in X , j in Y

Divide and Conquer

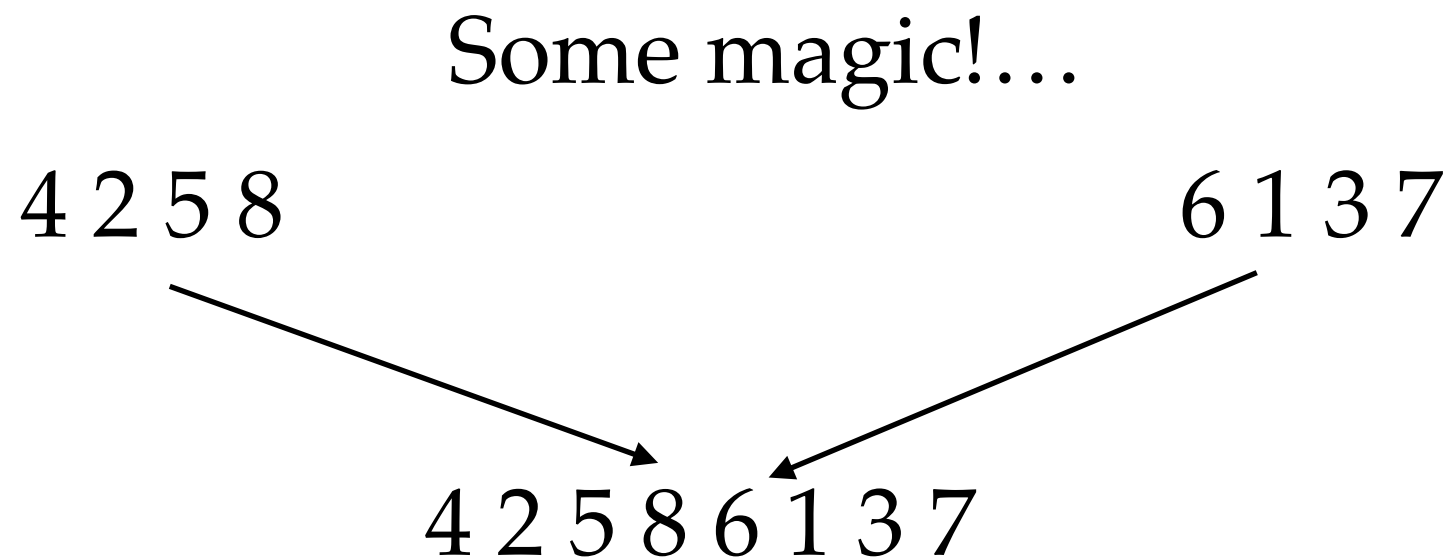
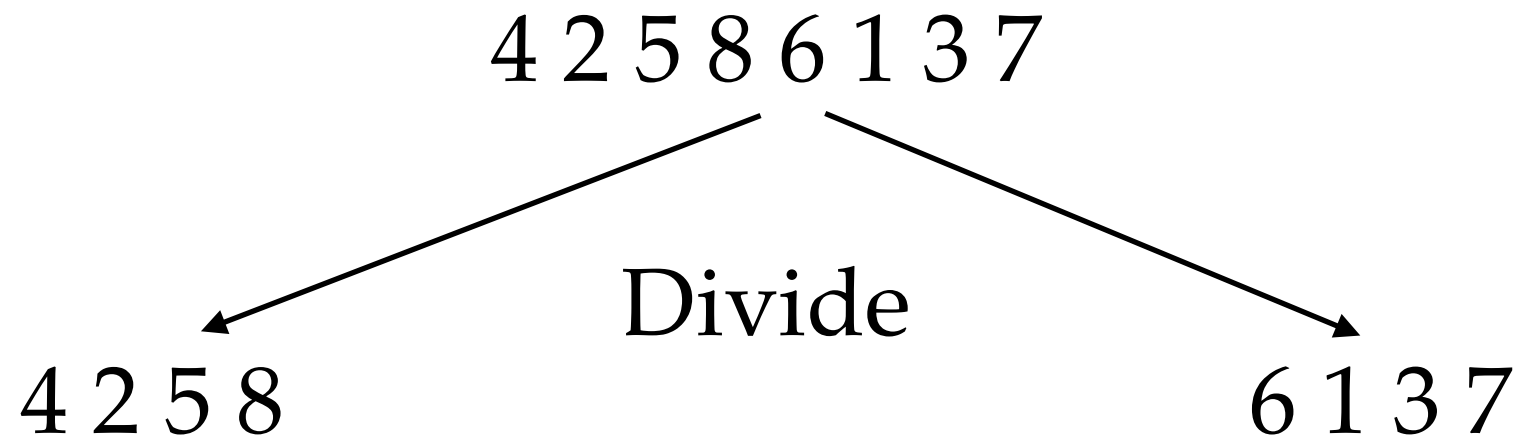
- Example: 1, 5, 3, 2, 4, 6
 - Left inversion : ($X(1), X(2)$)
 - Right inversions : None
 - Split inversions : $X(1), Y(0)$ $X(1), Y(1)$ and $X(2), Y(0)$

Divide and Conquer

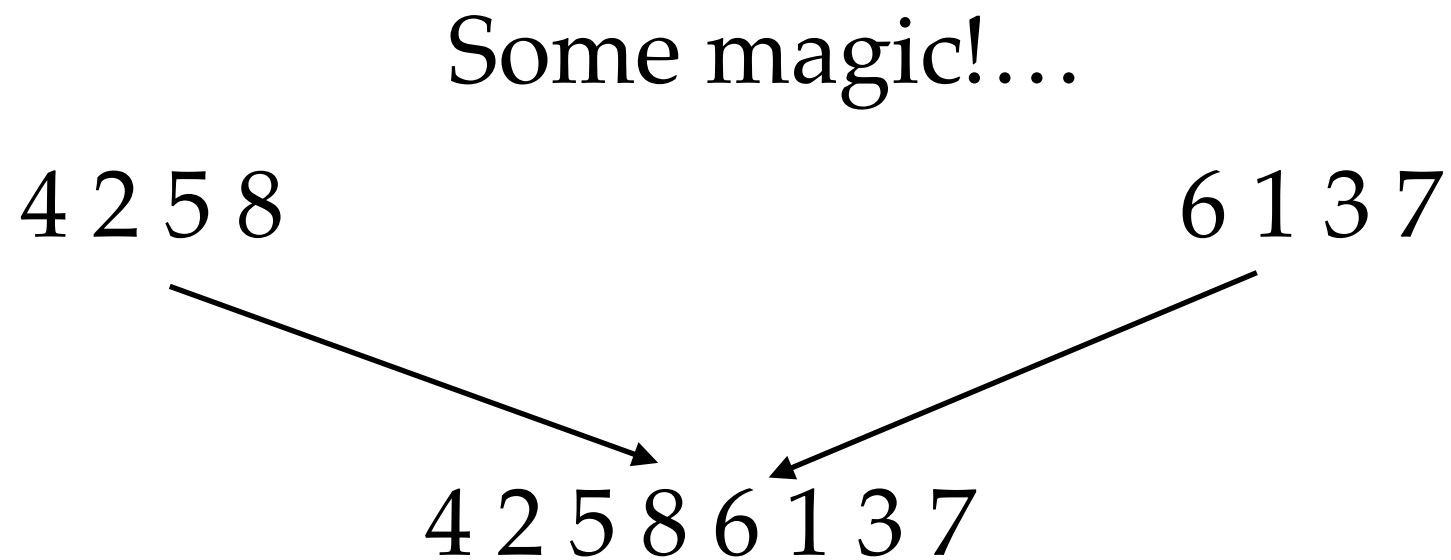
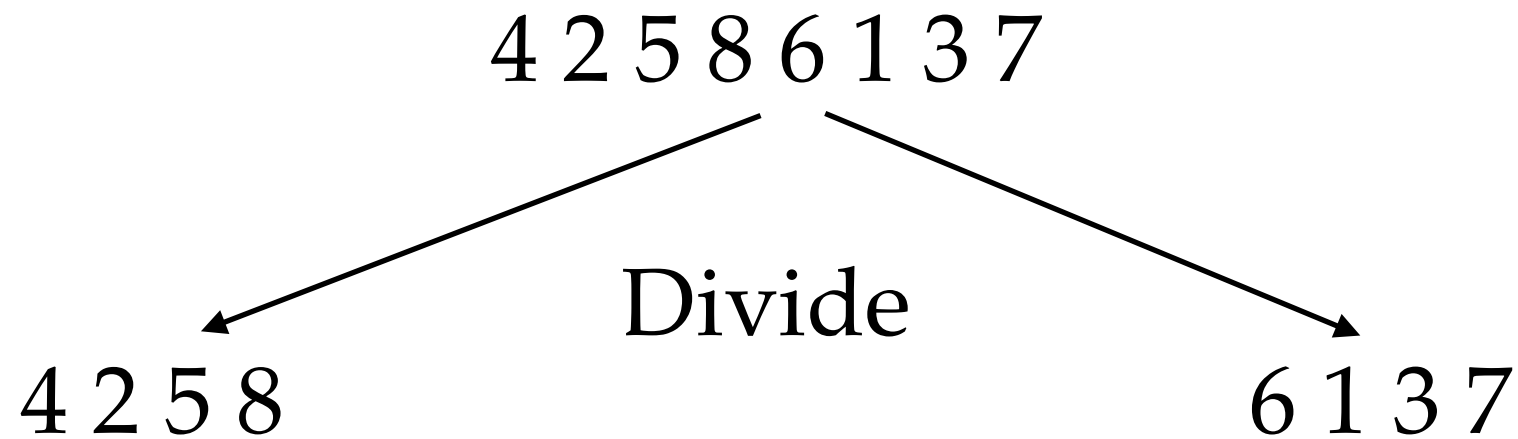
- Suppose A is divided in to X and Y .
- An inversion pair (i, j) is :
 - Left inversion : Both (i,j) in X
 - Right inversions : Both (i,j) in Y
 - Split inversions : i in X , j in Y

Algorithm: High-level idea

- CountInv (array A, length n)
 - If $n==1$ return 0
 - $X = A[1,2,\dots,n/2]$, $Y = A[n/2+1, \dots,n]$
 - $x = \text{CountInv}(X, n/2)$
 - $y = \text{CountInv}(Y, n/2)$
 - $z = \text{CountSplitInv}(X, Y, n)$
 - Return $x+y+z$



Count split inversion (conquer)



Count split inversion (conquer)

Q: How do we count split inversions in linear time?
- use Merge sort!...